ORF363/COS323 Introduction & Course Information (Lec1.pdf)

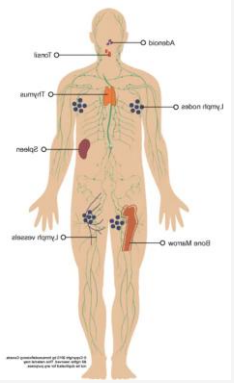
Ioannis Akrotirianakis

The definition and some examples

- **What is Optimization? (many definitions exist!)**
  - The process of finding the <u>best possible decision</u> from a (usually very large) set of alternatives, based on some <u>criteria of excellence</u> and <u>set of limitations</u>.
- Every living organism apply optimization for their survival and/or comfort!
  - **Birds** optimize their migration routes

  - **Honeybees** optimize the shape and size of honeycombs

  - **Humans** optimize the production of antibodies to fight bacterial infections



and many more…

Agenda for today

- Meet your teaching staff
- Meet your classmates
- Get your hands dirty with algorithms by playing 2 games
- Modeling with a Mathematical Program
  - Fermat's last theorem!
- Course logistics and expectations

Some small optimization problems you have already solved

- The classic student problem: What courses to take?
  - **Objective**: maximize learning, **or** maximize GPA, **or**…
  - **Constraints**:
    - Courses cannot have overlapping lectures;
    - No course before 10am;
    - Satisfy your desired certificate requirements

Some small optimization problems you have already solved

- What furniture to buy?
  - **Objective**: minimize cost, **or** maximize comfort, **or** …
  - **Constraints**:
    - Must fit your room
    - Must have 3 drawers
    - Should not be heavy

Some small optimization problems you have already solved

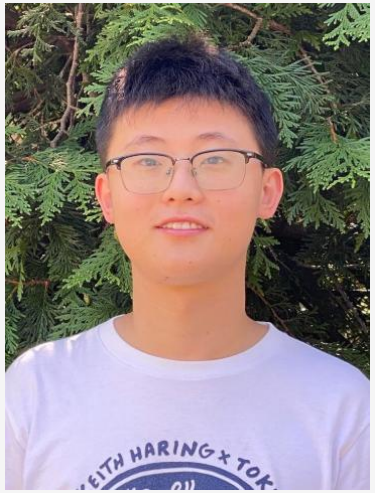
- What path to choose for my exercise today?
  - **Objective**: maximize encounter with other runners
  - **Constraints**:
    - Path must be between 5 and 7 miles
    - Chosen roads must have side walks and sea view
    - Path should get me back home (need to shower)

Meet your teaching staff

Ioannis Akrotirianakis
Lecturer at ORFE
Past life: Siemens, SAS, Princeton
Office Hours: TBD
Email: **iakrotir@p...**

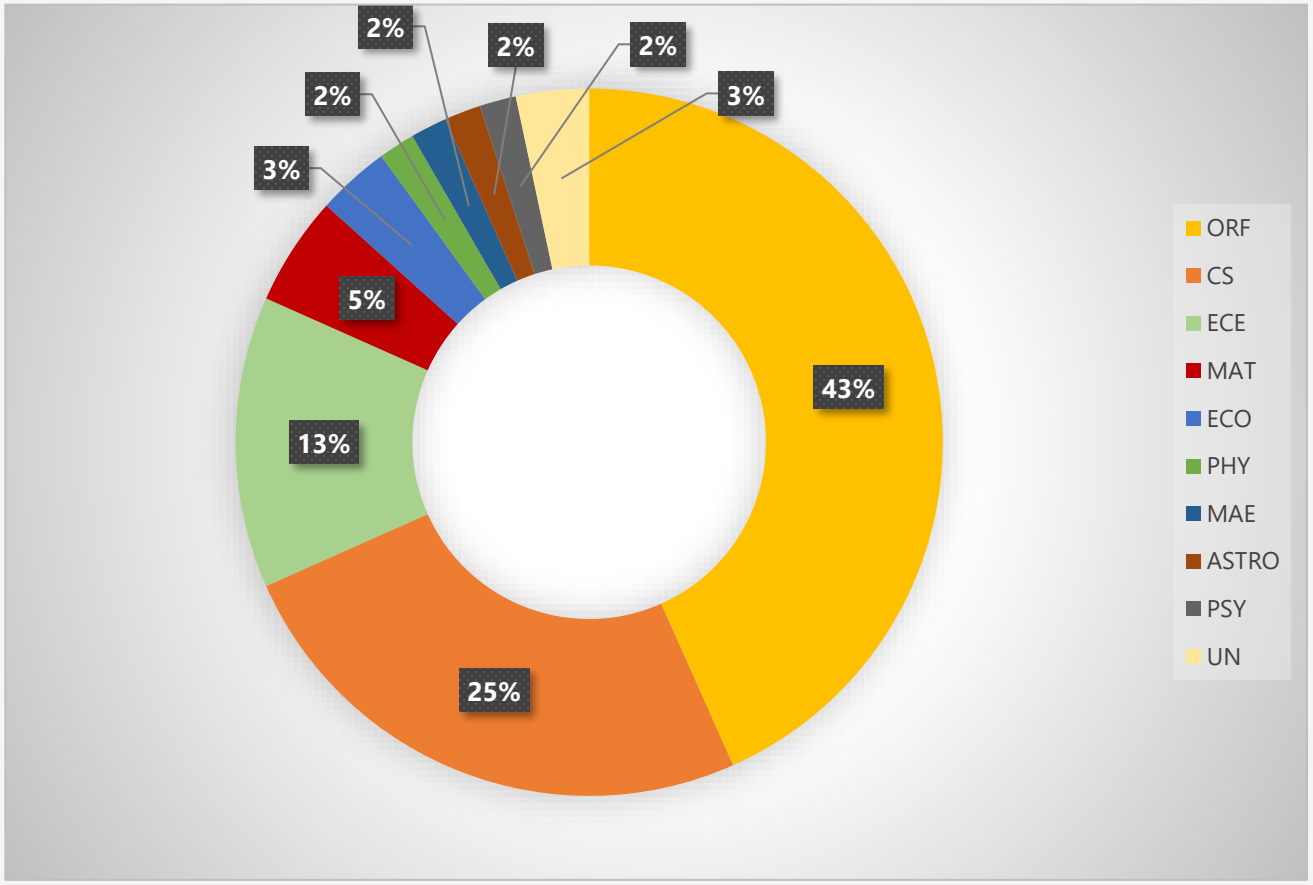
Burak Aydin
PhD student at ORFE
Office Hours: Wednesday
4:30-6:30, Sherrerd 122
Email: **ba7735@p...**

Jake Freeman
PhD student at ORFE
Office Hours: Mondays
4-6pm, Sherrerd 122
Email: **jake.freeman@p...**



Yukai Tang
PhD student at ORFE
Office Hours: Thursdays
1:30-3:30pm, Sherrerd 122
Email: **yt3846@p...**

Meet your classmates



Good mix of backgrounds!

Great opportunity for **collaborations**!

Course website

- http://aaa.Princeton.edu/orf363
- **Canvas** will contain
  - all lecture notes and slides,
  - **8** PSETS and their Solutions,
  - Syllabus, etc.
- Set up your notification setting on Canvas to receive our **announcements**!
- Use **EdDiscussion** to ask questions about the course

Check: `Canvas/Modules/`

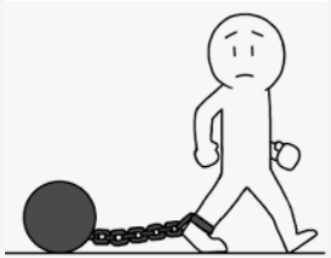
Let’s go back to Optimization…

*The **common theme** of every Optimization problem*

- You have (many) **decisions** to make

- You have an **objective (aka, goal or target)**, and you try to choose the decisions that maximizes or minimizes it

- You have **constraints** – your decisions must satisfy those constraints



What Optimization problems will be covered in this class

- We will deal with large optimization problems that have 1,000s (or even 1,000,000s) of decisions and constraints
- Such problems appear every day in scientific and engineering fields
- It is hopeless to make decisions in your head and with rules of thumb
- We need **mathematical techniques** that translate into **algorithms**
- We then translate these algorithms into **computer code** (Matlab, Python, …)

What Optimization problems will be covered in this class

- A powerful computer (your Mac, or Dell, or HP, …) will then execute the code and provide the **optimal decisions**
- The **mathematical description** of the decision problem is called the **Optimization Model**
- The **solution** is a vector containing the **optimal decisions**
- If we cannot reach the optimal solution, we would like to know an **approximate solution**

Some examples of Optimization problems

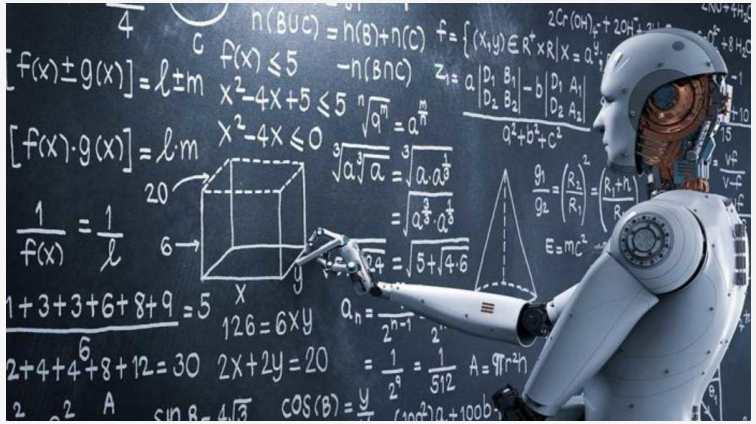
- **In Finance**
  - How to allocate your capital in 500 stocks”
    - **Objective:** maximize return OR minimize risk
    - **Constraints:**
      - No more than 40% of your money should be invested in one stock
      - Transaction costs should $\leq$ \$10,000
      - Return rate should be $\geq$ 8%

Some examples of Optimization problems

- **In Control Engineering:**
  - How to safely drive an autonomous vehicle from A to B?
    - **Objective:** minimize fuel consumption OR travel time
    - **Constraints:**
      - Distance to closest obstacle $\geq$ 3 meters
      - Vehicle speed $\leq$ 40 miles/hour
      - Path needs to be smooth (sudden turns are not allowed)

Examples of Optimization problems

- **In Machine Learning:**
  - How to identify whether an email is spam or not? (Binary Classification)
    - **Objective**: minimize probability of false negative
    - **Objective**: minimize overfitting
    - **Constraints:**
      - Probability of false positive ≤ 0.15
      - Misclassification error on training should be $\leq 5\%$



Computing and Optimization (ORF363/COS323)

- This class will give a broad introduction to **Optimization from a computational and algorithmic viewpoint**
- Optimization and Computing are very close areas of **Applied Mathematics**
- Several basic topics in **Scientific Computing** are either special cases or fundamental ingredients of more elaborate Optimization problems
  - **Examples**: Least squares, solving systems of linear equations and inequalities, matrix factorizations, approximation and fitting, conjugate gradients, ….

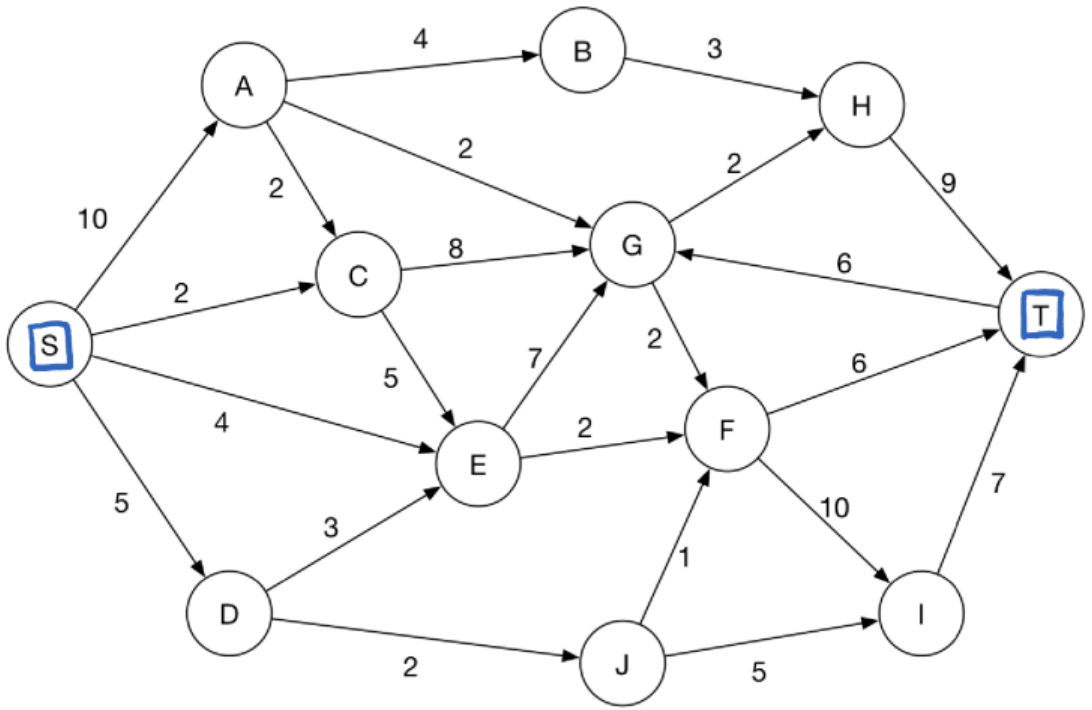
Time to play 2 Games ☺

Game #1: Max-Flow

Game #1: Max-Flow

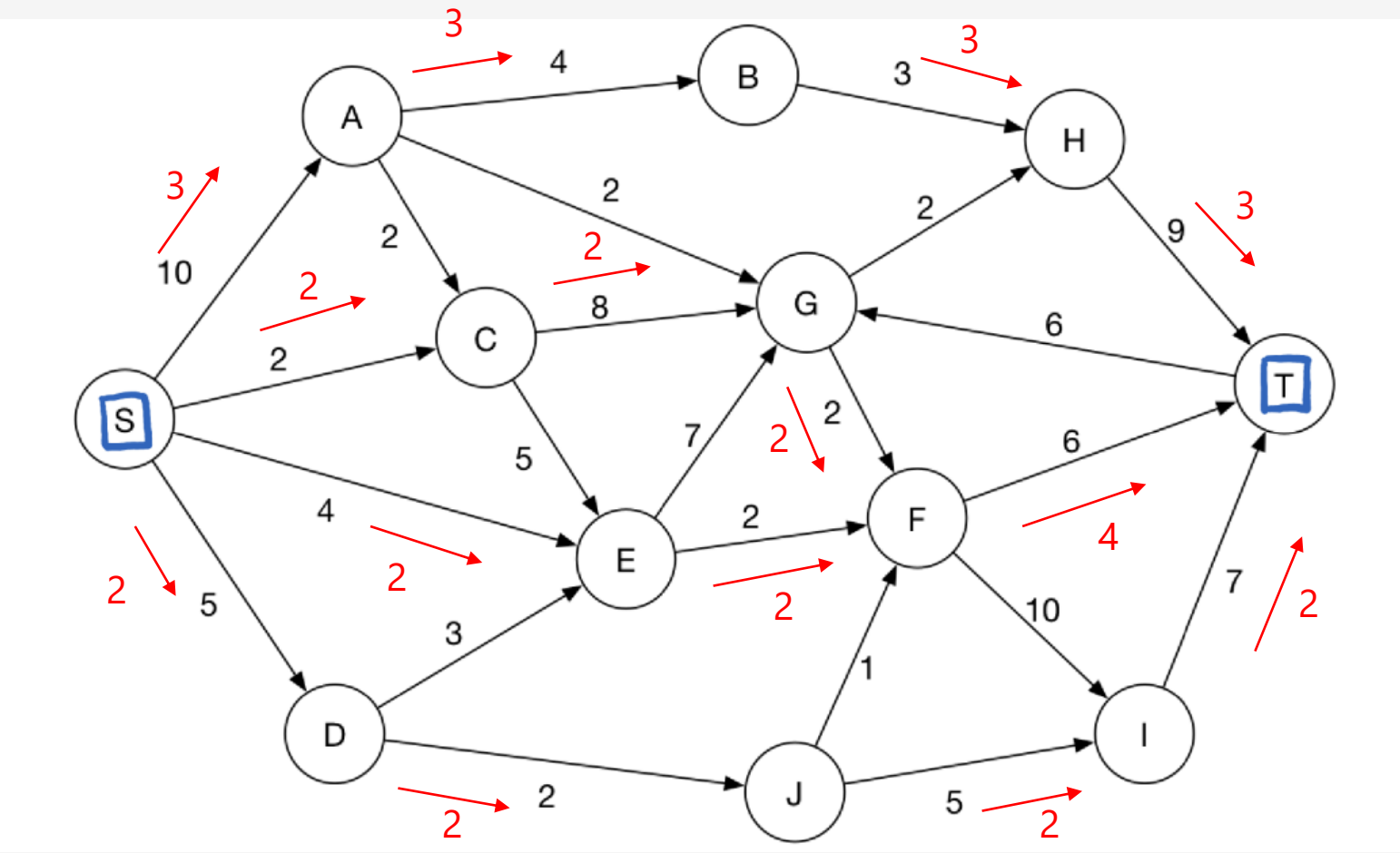
- **Your goal (objective):**
  - **Ship the maximum amount of oil from S to T**
- **Rules of the game (constraints):**
  - Cannot exceed capacity on the edges
  - For each node, except for S and T:
    - InFlow == OutFlow

(no storage at the nodes…)



Game #1 : Max-Flow

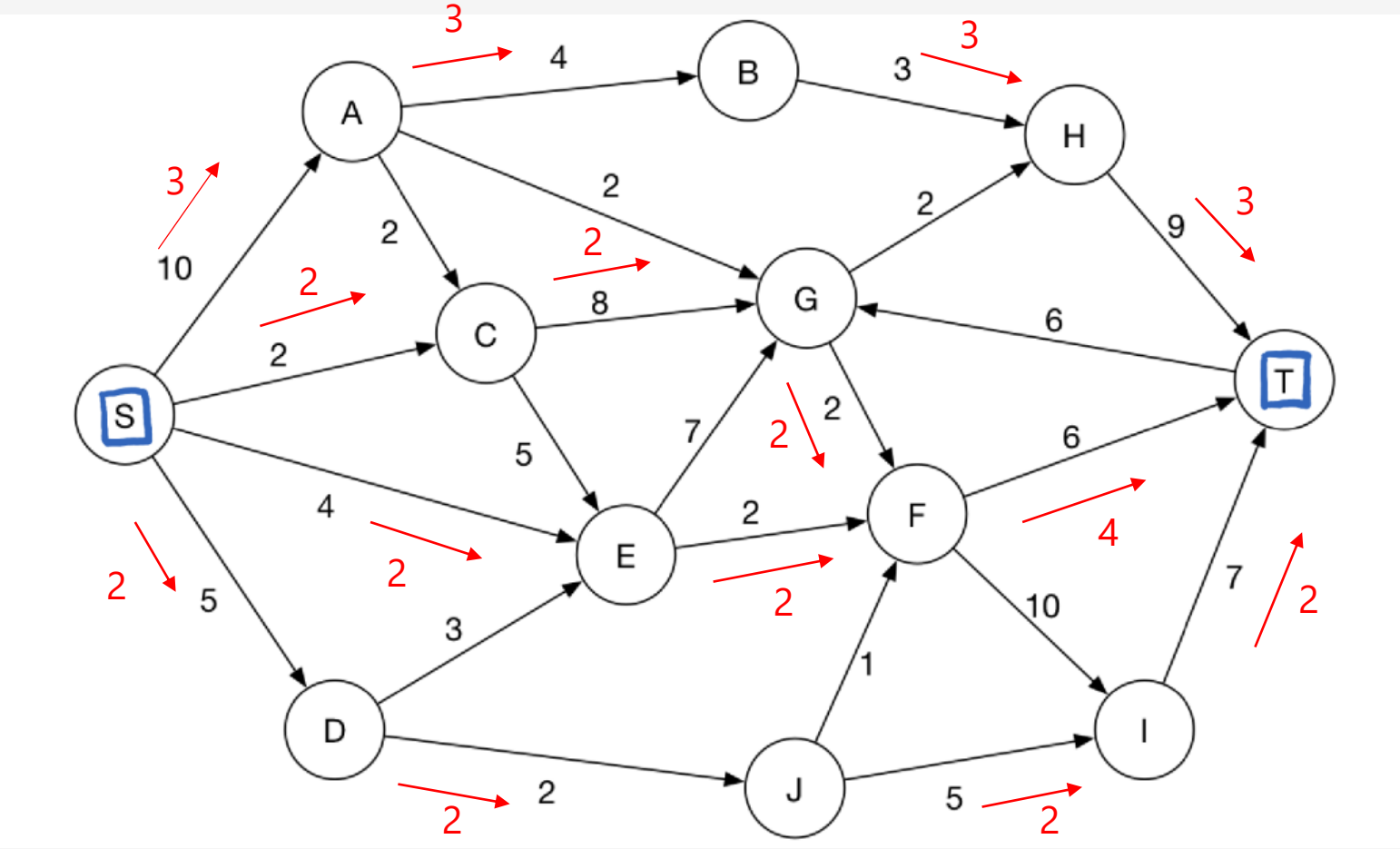
- Here is an example:



Flow value = 3+4+2 = 9

Game #1 : Max-Flow

- Here is an example:



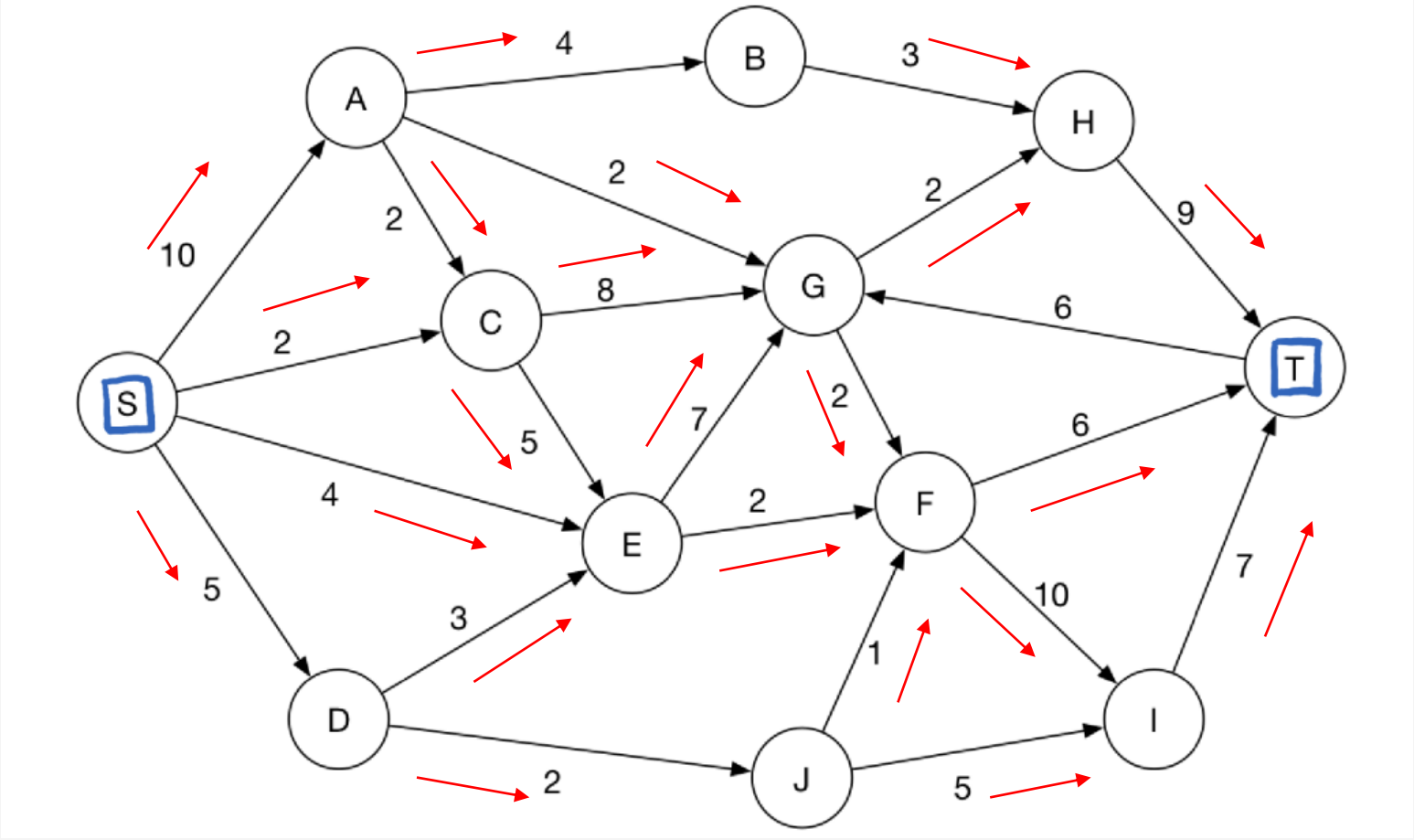
- **Can you do better?**
- **How much better?**

- Write your solution on the paper you were given
- You have 5 minutes!

Flow value = 3+4+2 = 9

Game #1 : Max-Flow

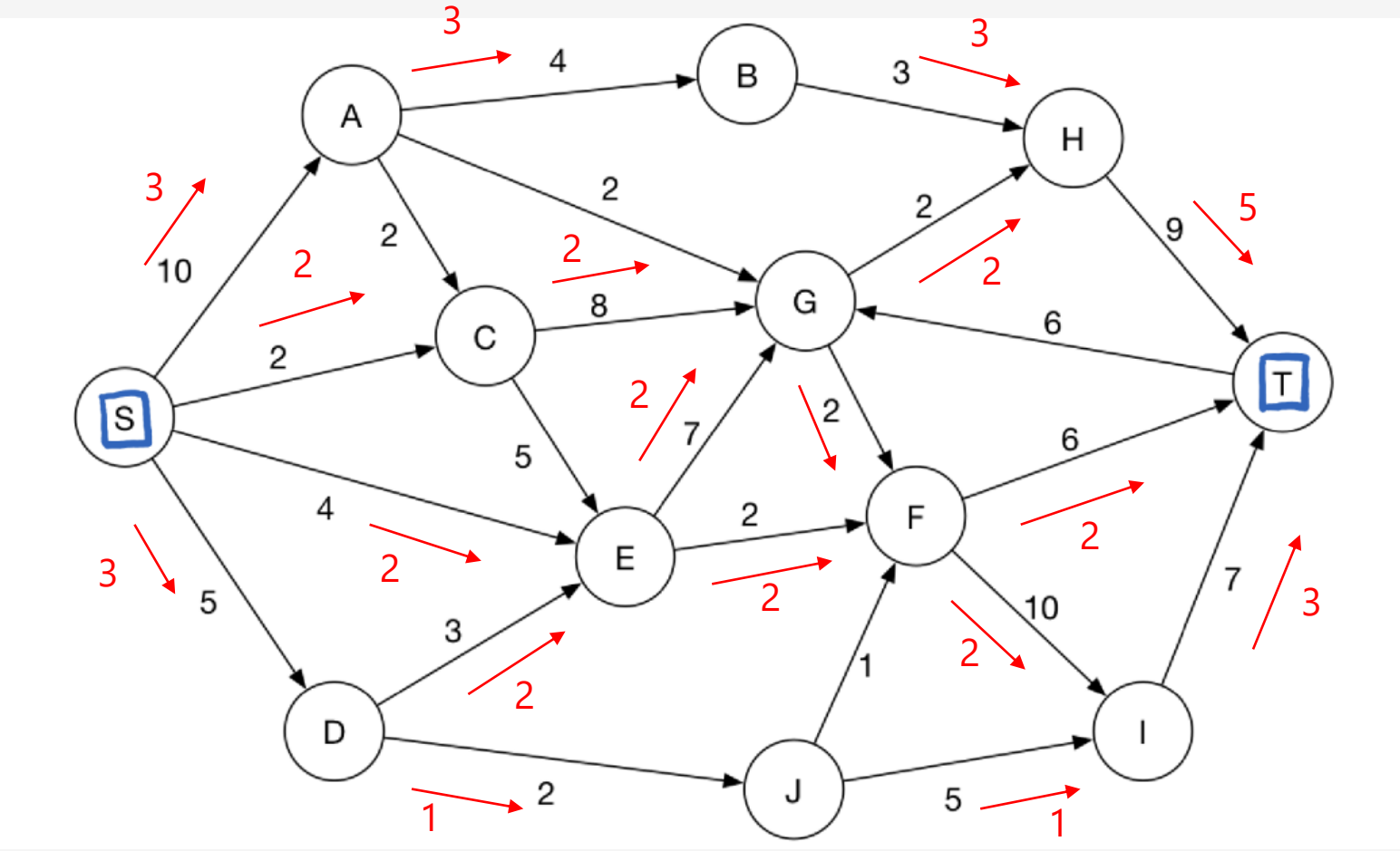
- Tell me your solution…



Flow value = ??

Game #1 : Max-Flow

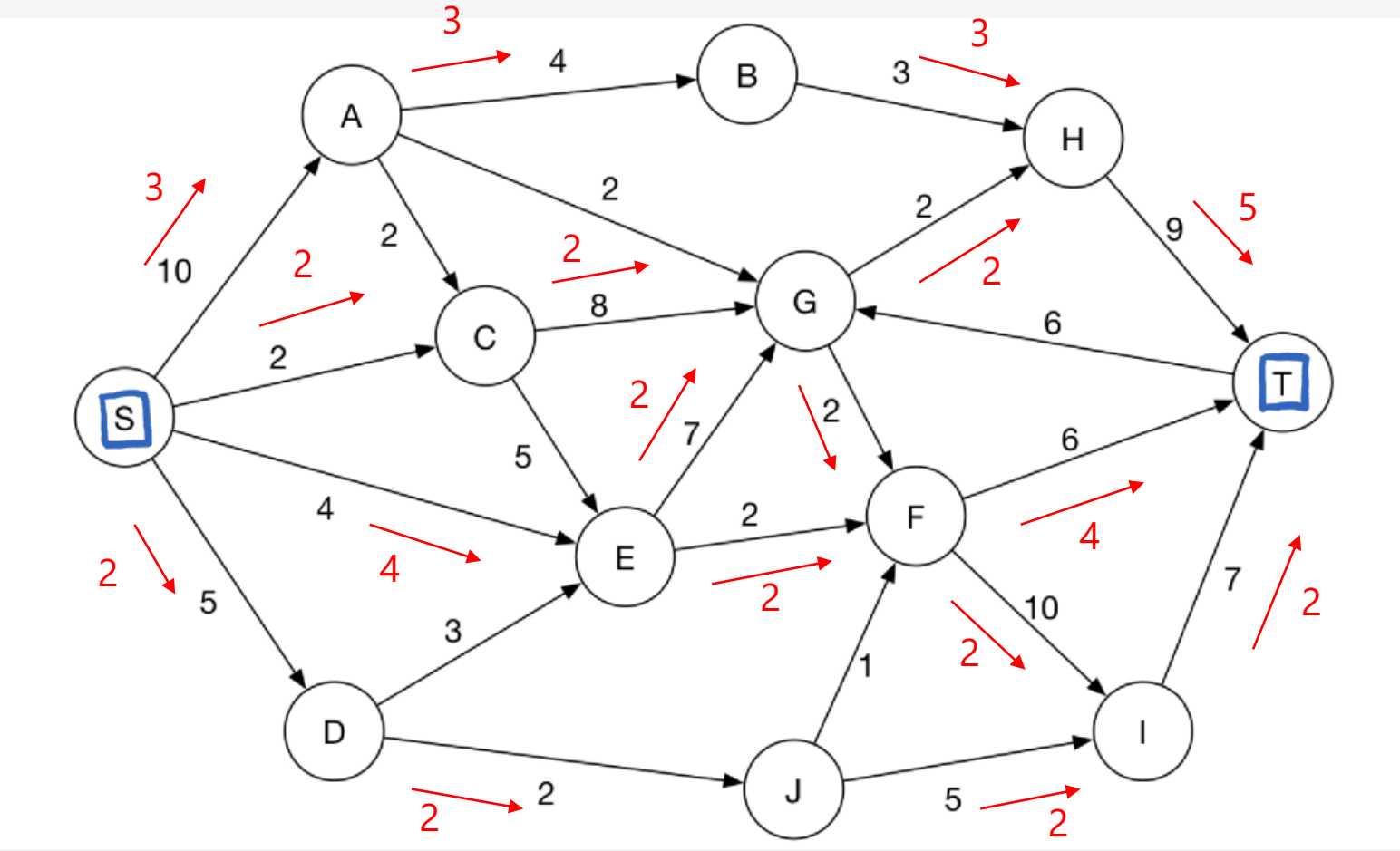
- A couple of other good solutions:



Flow value = 5+2+3 = 10

Game #1 : Max-Flow

- A couple of other good solutions:



Flow value = 5+4+2 = 11

Game #1 : Max-Flow

Can the flow be more than 11?

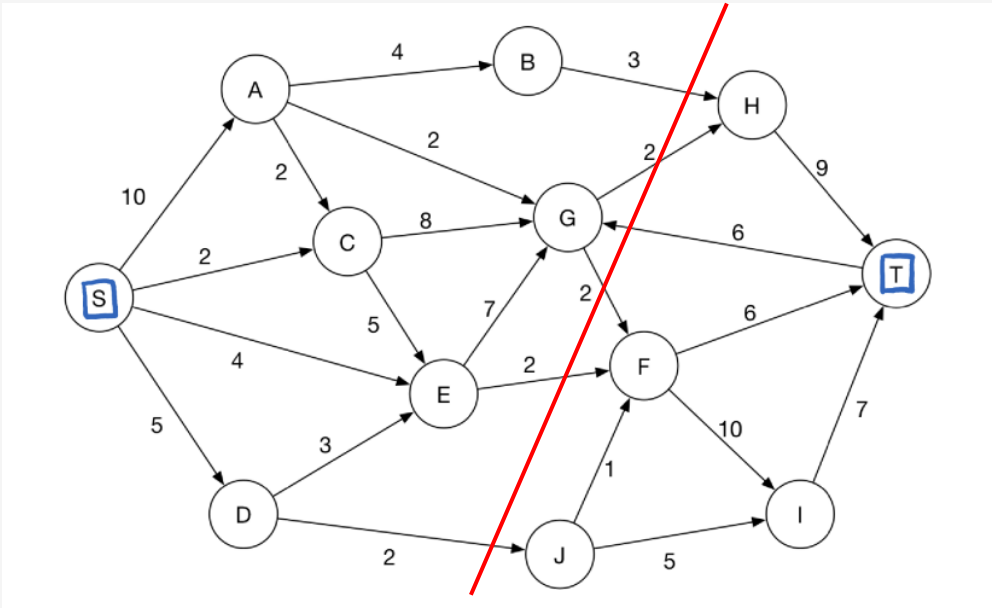
Is 11 the optimal value?

Game #1: Max-Flow

- **11 is the optimal solution** because…
  - Any flow from S to T must cross the red line (aka min-cut)…
    - whose total flow is 11
    - So, 11 is the optimal flow

The min-cut proves optimality

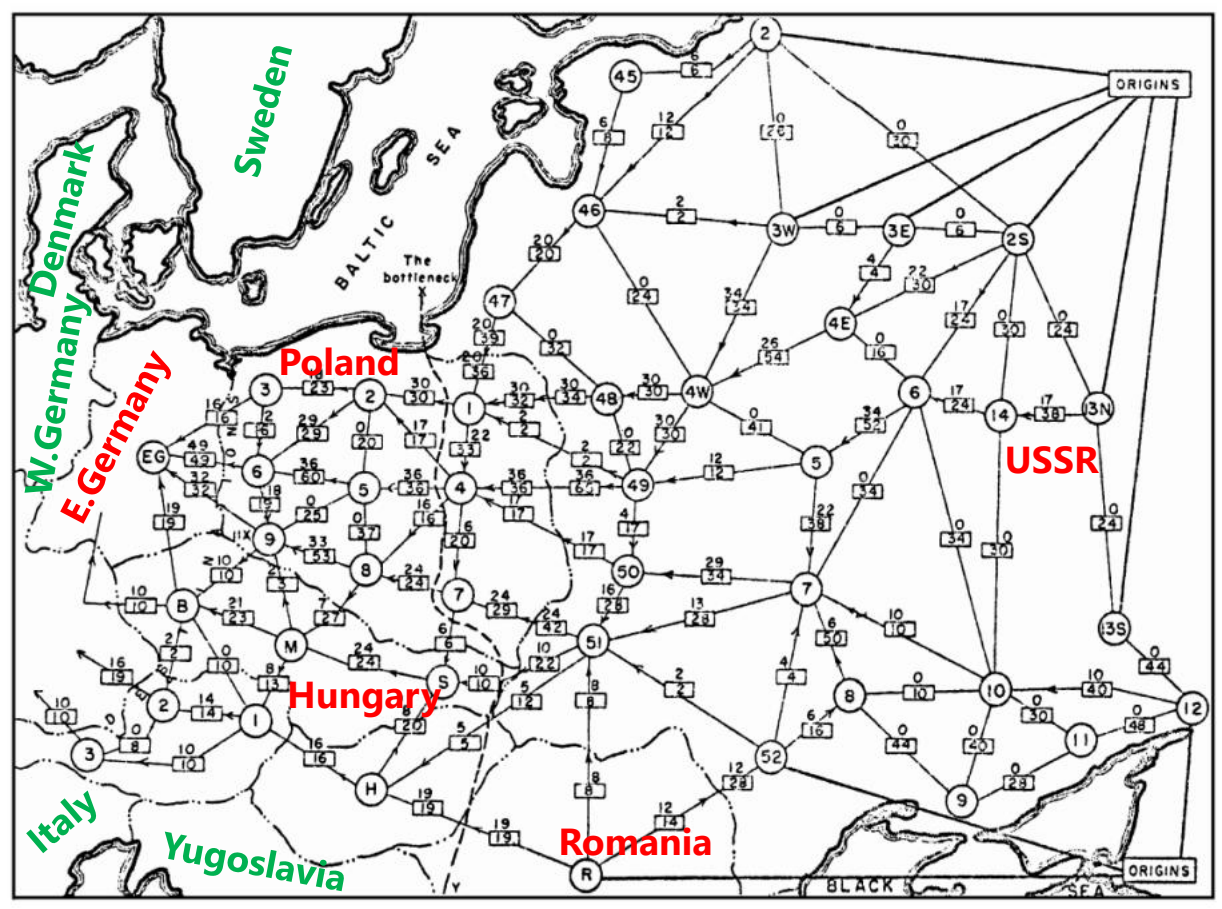
(proof by magic…)



$3 + 2 + 2 + 2 + 2 = 11$

Game #1: Max-Flow

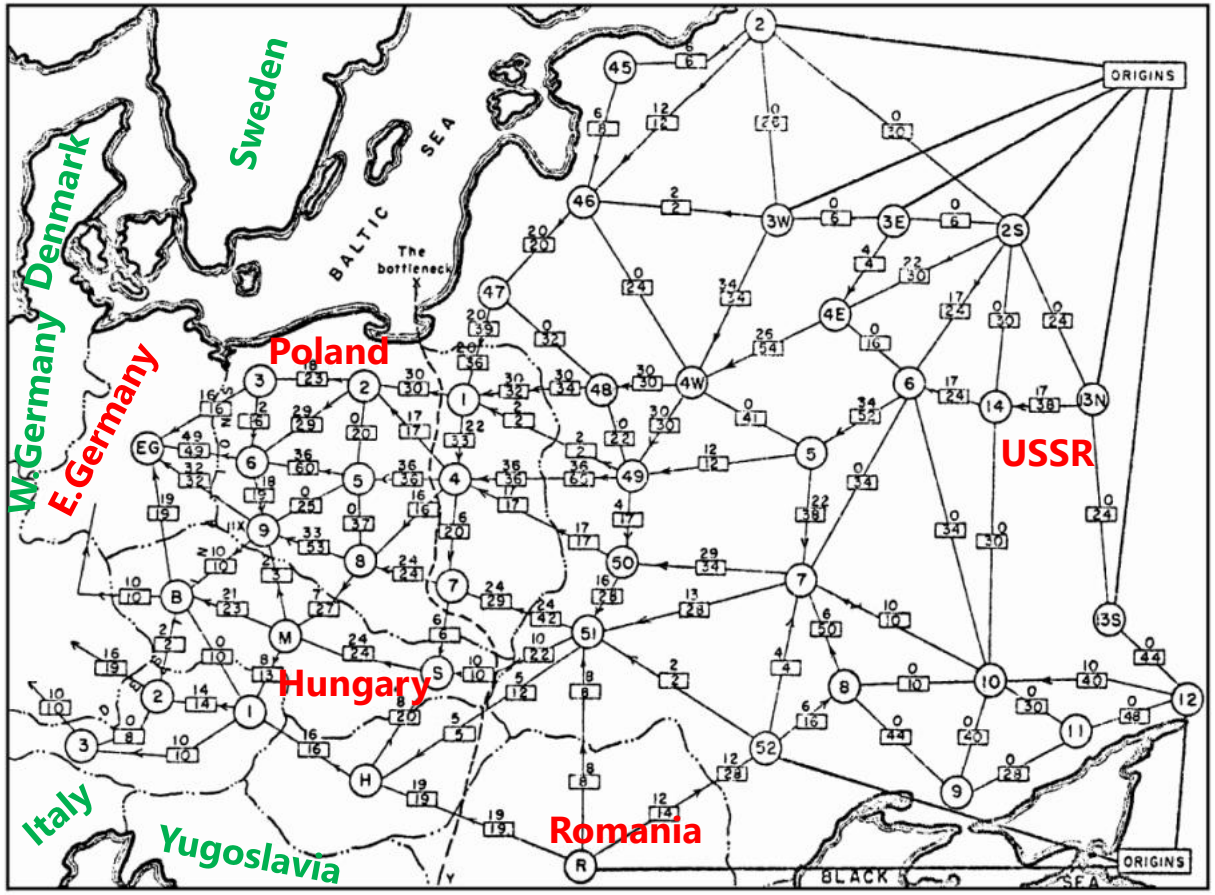
- Lets' try a more *"slightly larger"* graph …



You have 5 minutes… ☺

Game #1: Max-Flow

- How would someone who has not seen optimization approach it?
  - Trial & error?
  - Do they have any chance to solve it?
- It is **futile** to try to solve this problem by hand!! May take months to solve it…
- Optimization Algorithms can solve it very fast on a conventional computer (Mac/Dell…)



Game #1: Max-Flow

From a secret **USAF** report by Harris & Ross (1955).

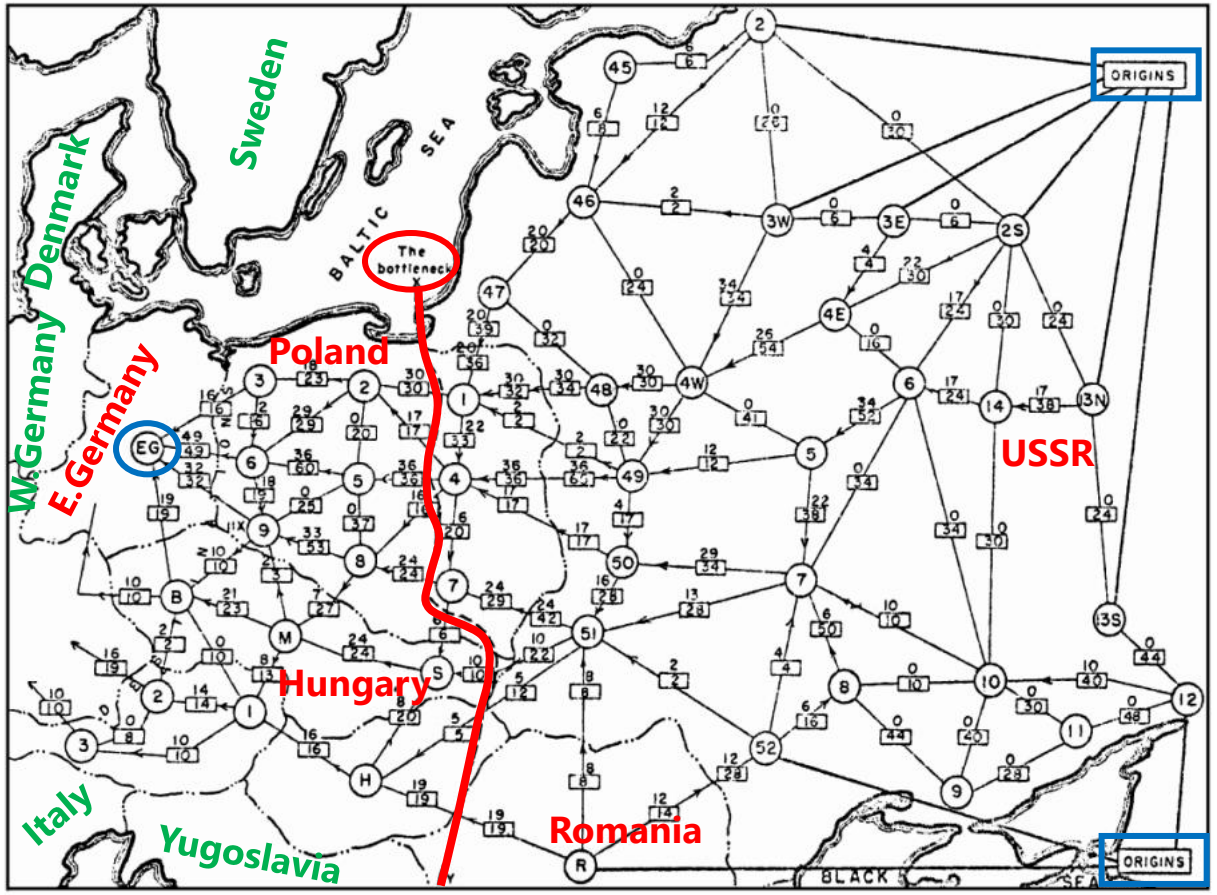
Shows the railway network of western **USSR** going to eastern Europe.

Declassified in 1999

Look for the min-cut on the map (aka, 'bottleneck')

There are 44 vertices, 105 edges, and the max-flow is 163.

Harris & Ross gave a **heuristic** which happened to solve the problem **optimally** (lucky for this graph…)



Soon after that, the famous **Ford-Fulkerson algorithm** came out of the RAND Corporation.
That algorithm always finds the **optimal solution for rational edge costs**.

See [Sch05], for more **history** on this problem

Game #2: Exam scheduling

Game #2: Exam scheduling

- The department chair of ORFE would like to **schedule the final exams** for **12 graduate courses** offered this semester.

- **Objective:** have **as many exams as possible on the same day**, so everyone
- gets done quickly and goes on vacation.

- There is just one **constraint:**
  No student should have > 1 exam on the same day.

How can we model this Optimization problem as a **Graph**??

Game #2: Exam scheduling

- The department chair of ORFE would like to **schedule the final exams** for **12 graduate courses** offered this semester.

- **Objective:** have **as many exams as possible on the same day**, so everyone
- gets done quickly and goes on vacation.

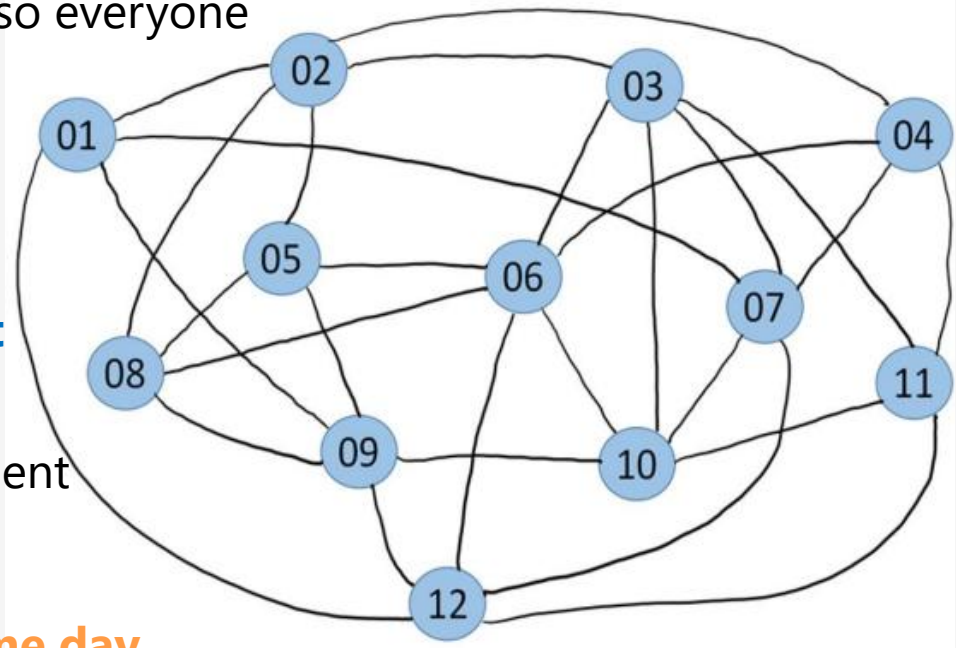
- There is just one **constraint:**
  No student should have > 1 exam on the same day.

- The **nodes** of this graph are the 12 courses.

- There is an **edge** between two nodes iff there is at least one student who is taking both courses.

- **If we want to schedule as many exams as possible on the same day, what are we looking for in the graph ??**

Represent it as a Graph



Game #2: Exam scheduling

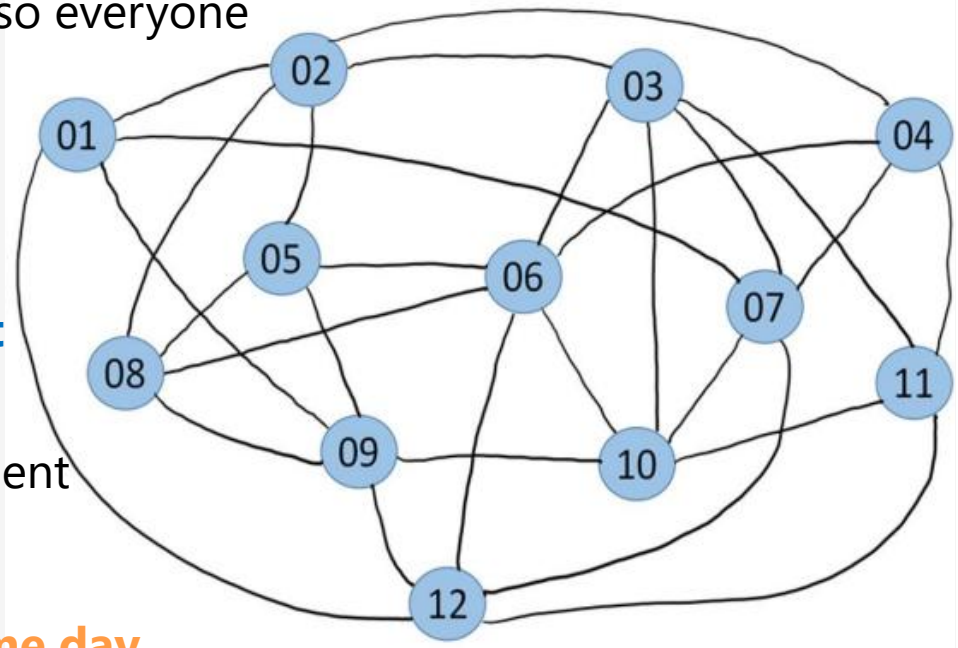
- The department chair of ORFE would like to **schedule the final exams** for **12 graduate courses** offered
- this semester.

- **Objective:** have **as many exams as possible on the same day**, so everyone
- gets done quickly and goes on vacation.

- There is just one **constraint:**
  No student should have > 1 exam on the same day.

- The **nodes** of this graph are the 12 courses.

Represent it as a Graph



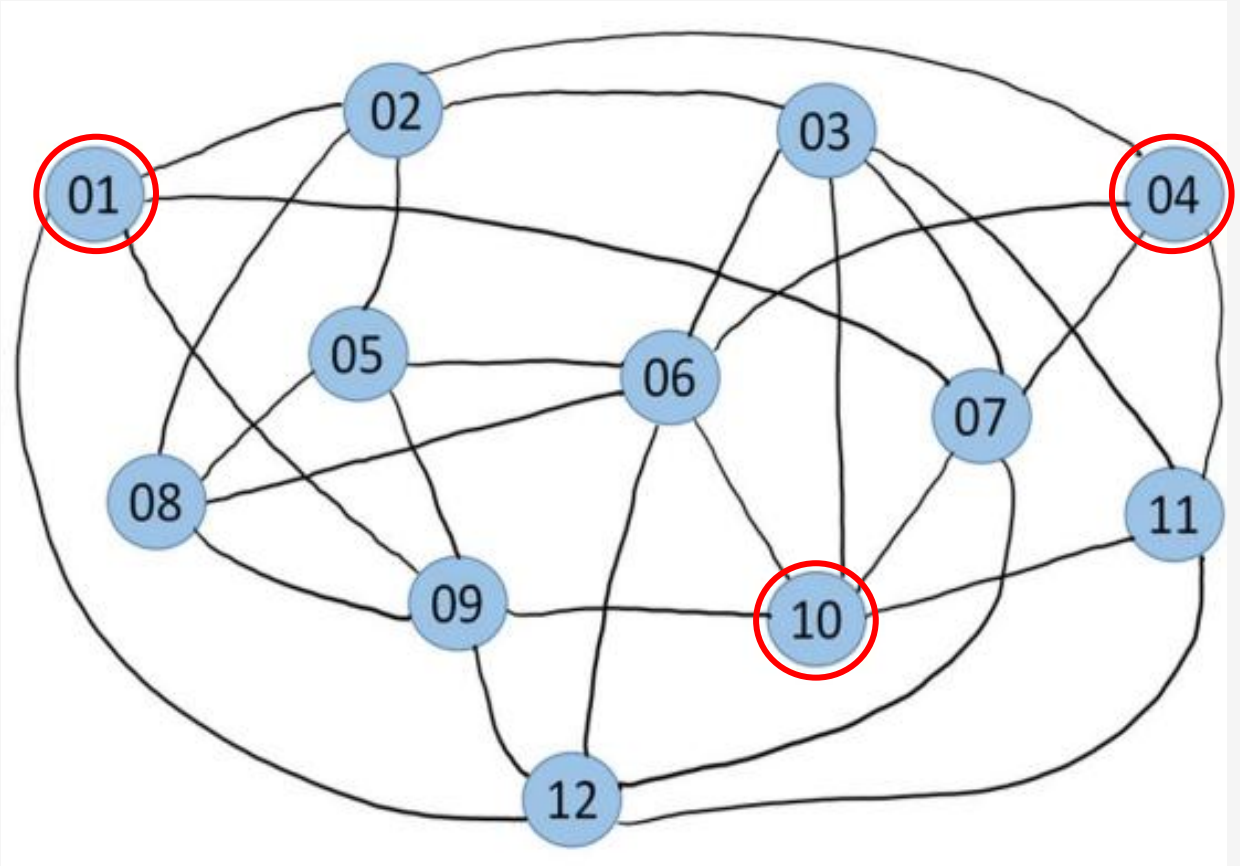
- There is an **edge** between two nodes iff there is at least one student who is taking both courses.

- **If we want to schedule as many exams as possible on the same day, what are we looking for in the graph ??**

- **Equivalent Objective: Find the largest collection of nodes such that no two nodes share an edge**

Game #2: Exam scheduling

Here is a possible solution



Game #2: Exam scheduling

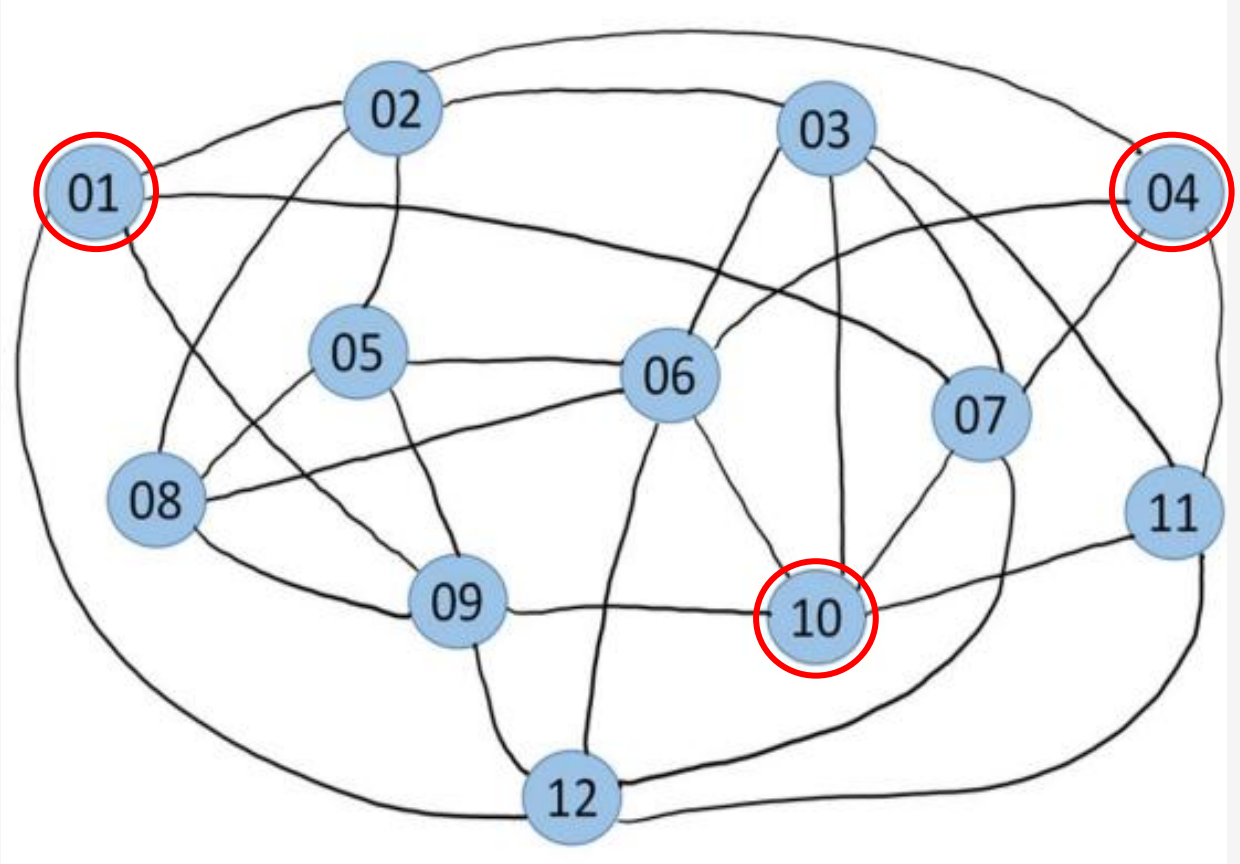
Here is a possible solution

Can you do better than 3 courses?

How much better?

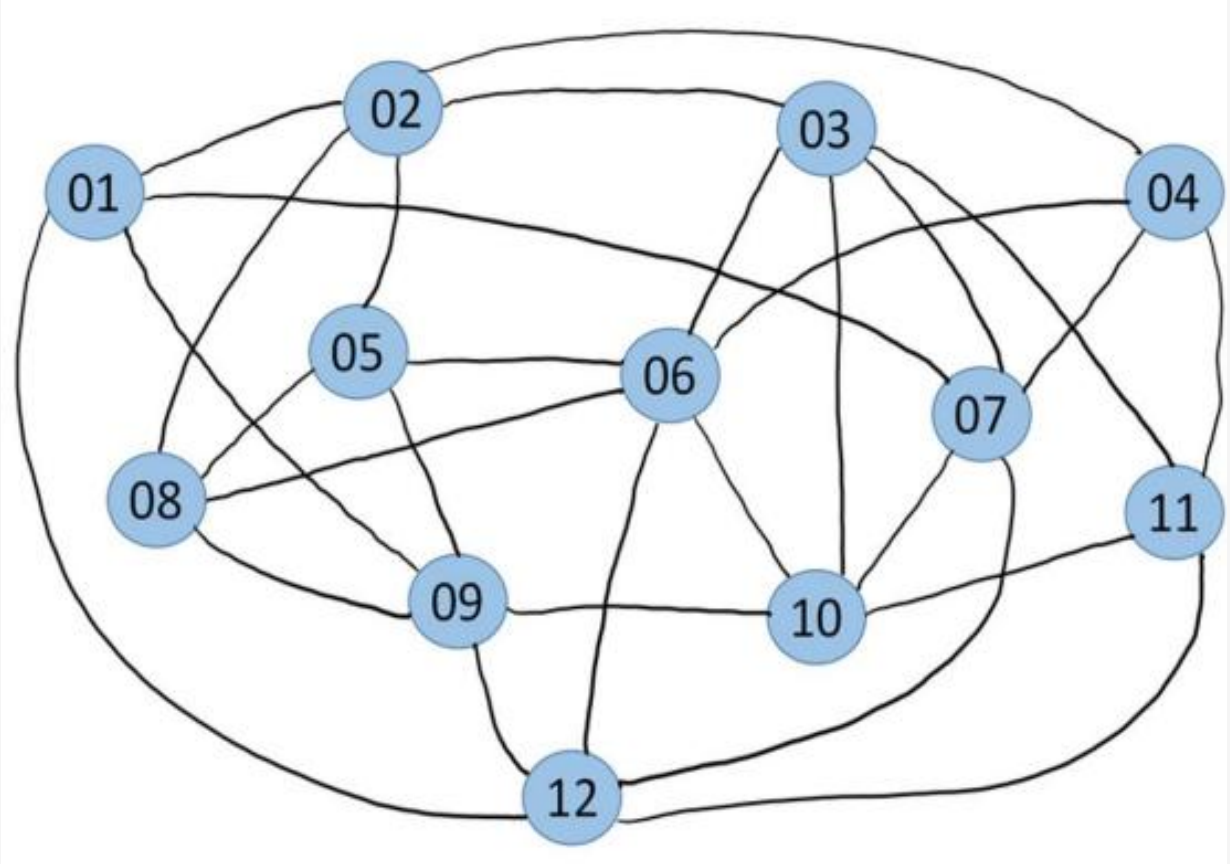
Write your solution on the paper.

You have 5 minutes!



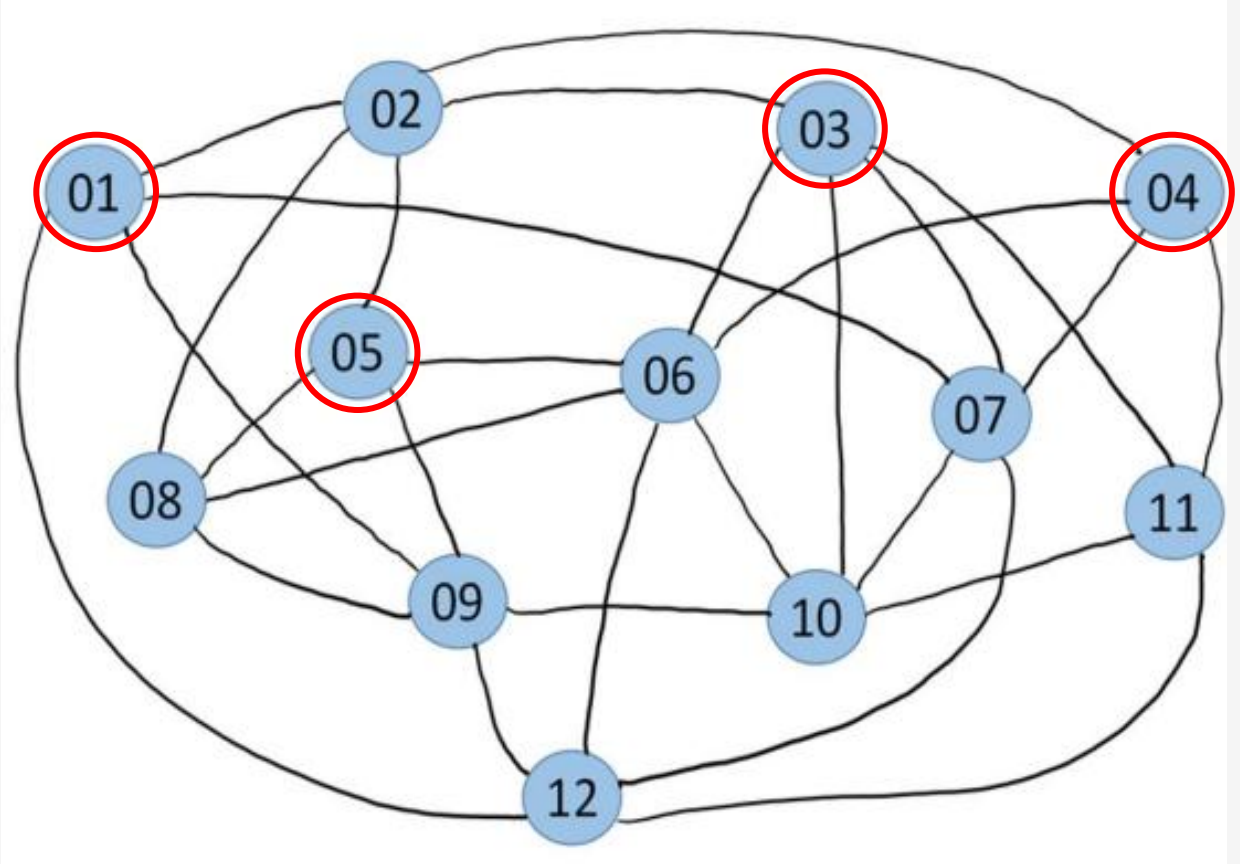
Game #2: Exam scheduling

- Tell me your solution…



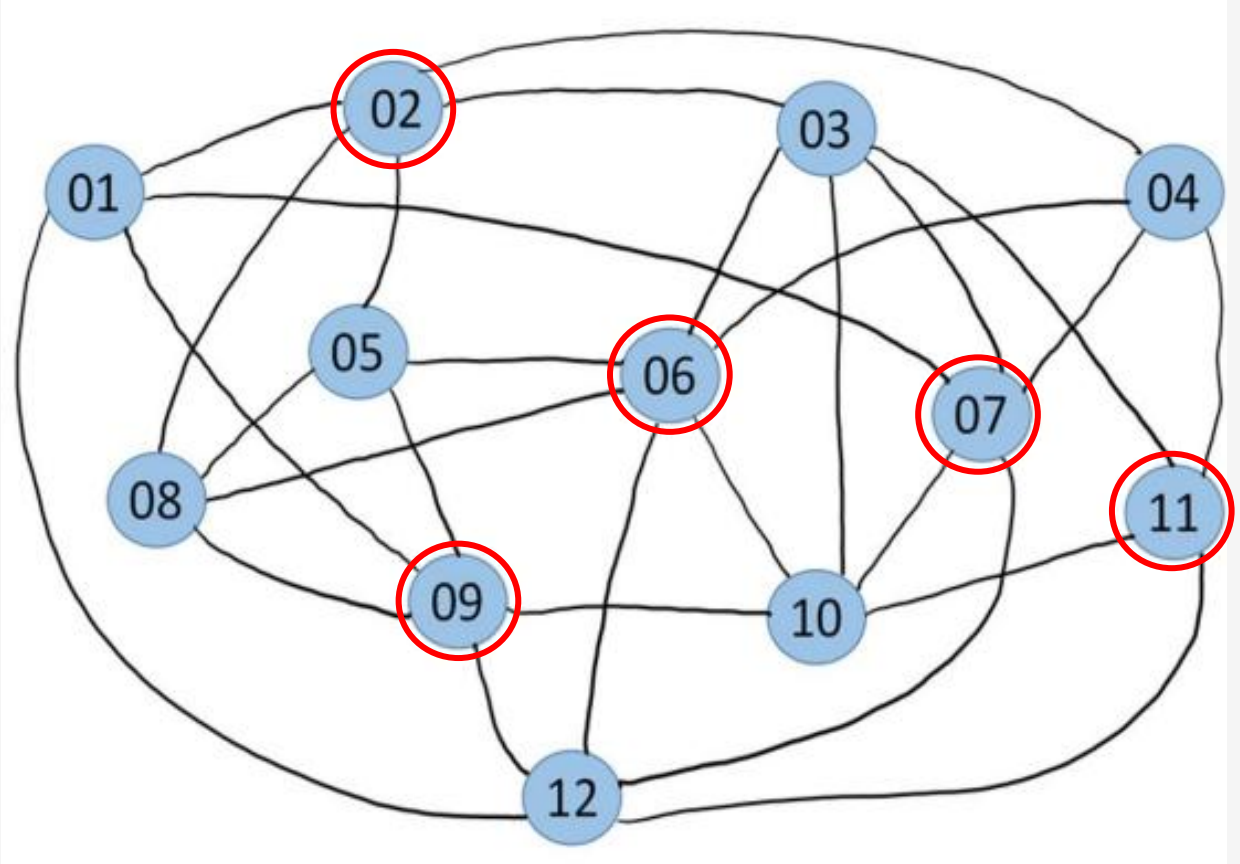
Game #2: Exam scheduling

4 courses scheduled on the same day



Game #2: Exam scheduling

5 courses scheduled on the same day



Game #2: Exam scheduling

Can we schedule more than 5 courses?

Is 5 the optimal value?

Game #2: Exam scheduling

Proof by magic?

Unfortunately, not ☹

No magician in the world has pulled out such a rabbit to this day.!
(by this we mean a trick that would work on all graphs)

Of course, there is always a proof:

Brute force → Try all possible subsets containing 6 nodes.
Can take long time, since there are $\binom{12}{6} = 924$ combinations

But this is no magic. It impresses nobody.

We want a 'short' proof!
Like the one in our max-flow example.

Let's appreciate this further…

Game #2: Exam scheduling

Let's try another graph.

Encouraged by the success of ORFE, now the Dean of Engineering Wants to do the same for **115 SEAS courses**.

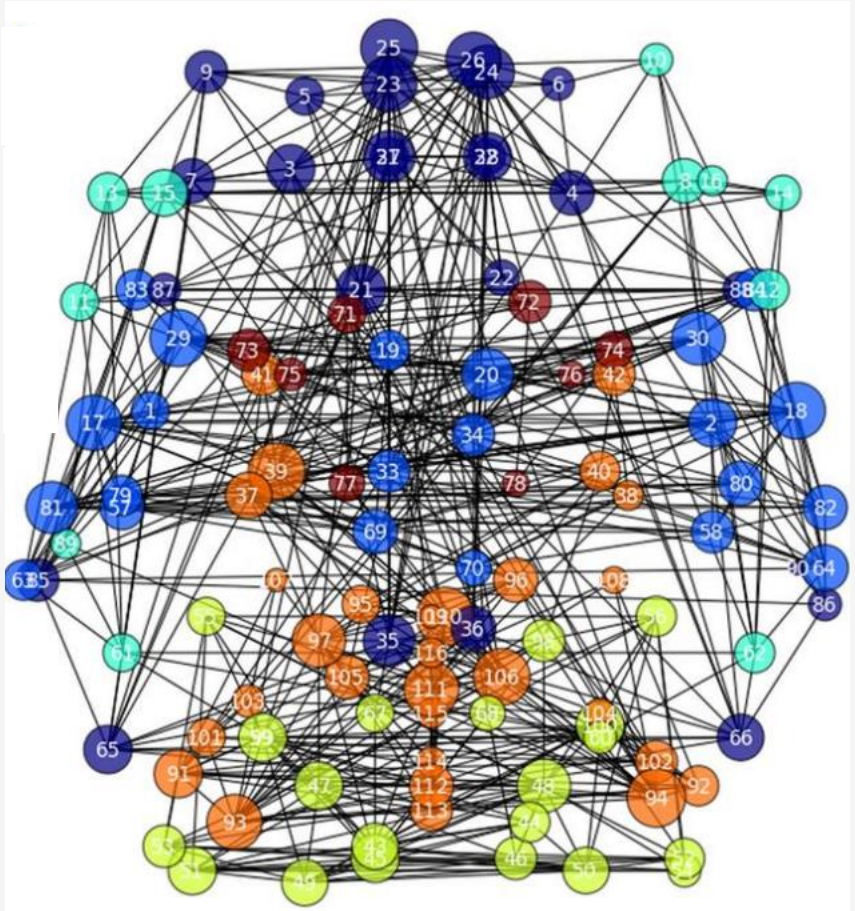
How many final exams on the same day are possible?

Question: Can you schedule 17 courses on the same day?

Want to try out all possibilities for 17 exams?

You will get over 8×10^{19} of them!

You need more than 5 minutes to solve this…



Game #2: Exam scheduling

But there are some good news

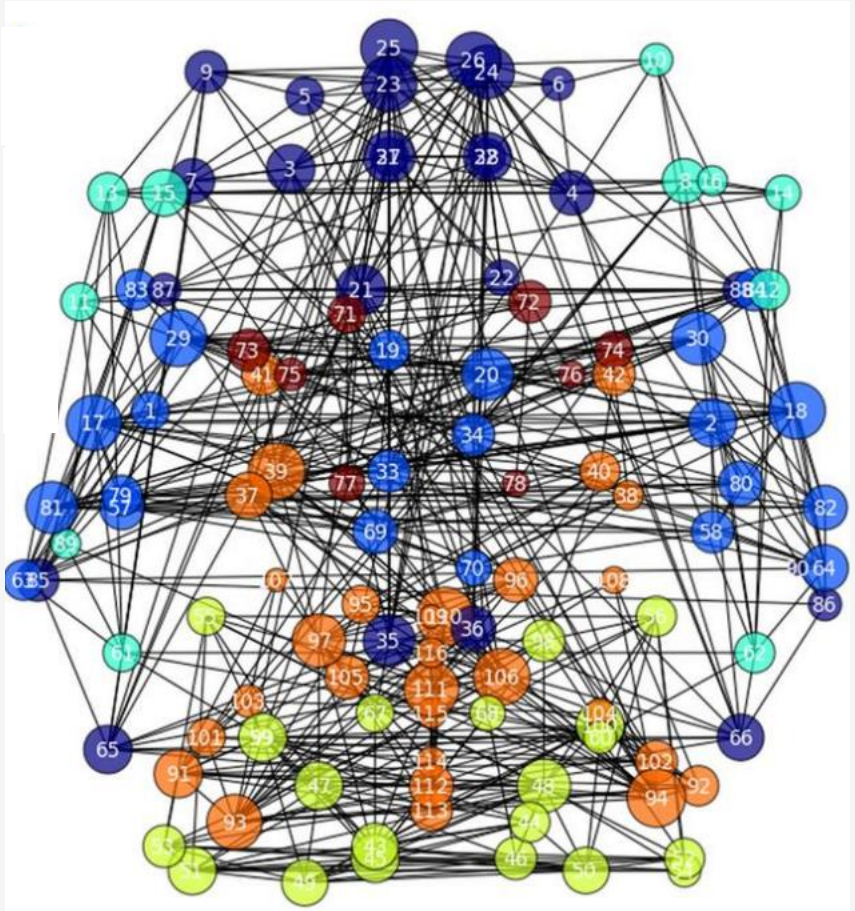
We will learn powerful techniques from **Optimization**, and in particular, from the area of **Convex Optimization**, that allow us to **find high-quality solutions** with performance guarantees.

For example, an optimization algorithm may quickly find **16 possible concurrent exams.**

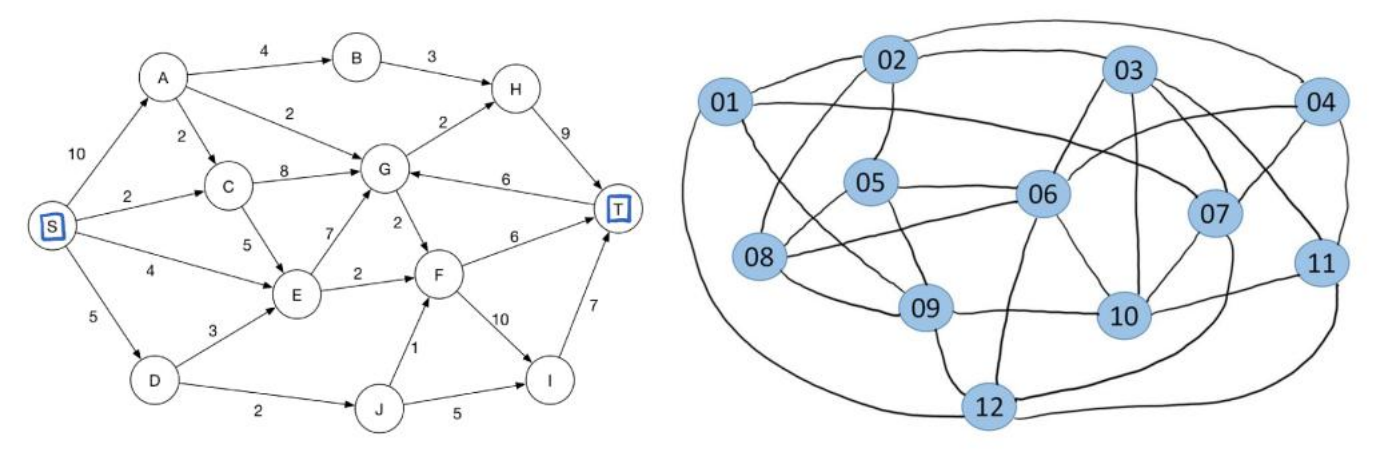
Another optimization algorithm (or sometimes the same one) tells you that **19 is impossible.**

This is very useful information!

you have trapped the optimal solution within 16 and 19



Which game was harder for you?



Not always obvious! A lot of research in Optimization and Computer Science goes into distinguishing the **"tractable"** problems from the **"intractable"** ones.

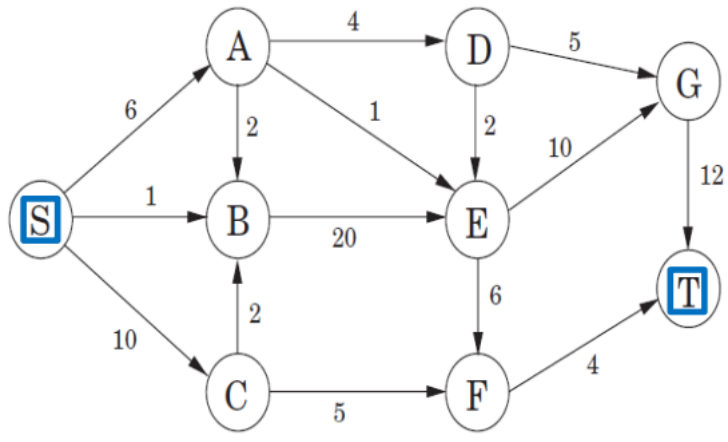
The two brain teasers actually just gave you a taste of the P vs NP problem.

<u>The first problem</u> (max-flow) we can solve efficiently (in "polynomial time")

<u>The second problem</u>: no one knows! If you do, you literally get **$1,000,000**
More importantly, your algorithm will solve many more other hard problems, and you will become famous!

Modeling problems as Mathematical Programs

Let's revisit our first example – Max-Flow



What were your **decision variables**?

What were your **constraints**?

What was your **objective function**?

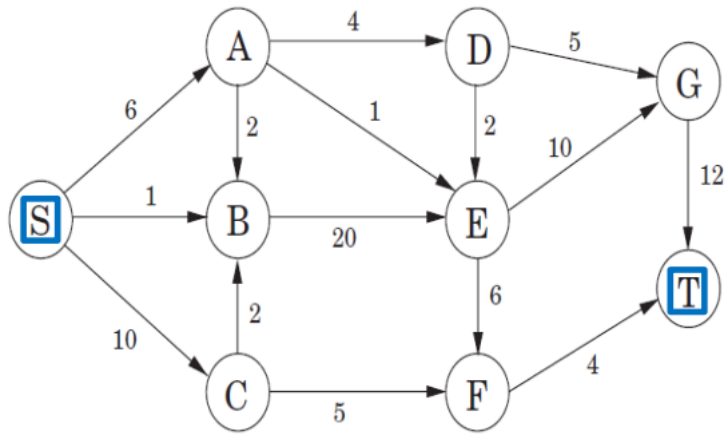
Rules of the game:

cannot exceed capacity on the edges

for each node, must have: InFlow == OutFlow (except for S and T)

Goal: ship as much oil as you can from S to T.

Let's revisit our first example – Max-Flow



Think of the transportation network as a **graph**: $G = (V, E)$

$$V = \{S, A, B, C, D, E, F, G, T\}$$
$$E = \{(S, A), (S, B), \dots, (G, T), (F, T\}$$

Decision variables:

x_{ij}: amount of oil transported from node i to node j for all $(i, j) \in E$

Objective: maximize the oil coming out from node S

$$\max\ (x_{SA} + x_{SB} + x_{SC})$$

Constraints:

<u>balance constraints → InFlow == OutFlow</u>

$$x_{SA} = x_{AD} + x_{AE} + x_{AB}$$
$$x_{AD} = x_{DG} + x_{DE}$$
$$\dots$$

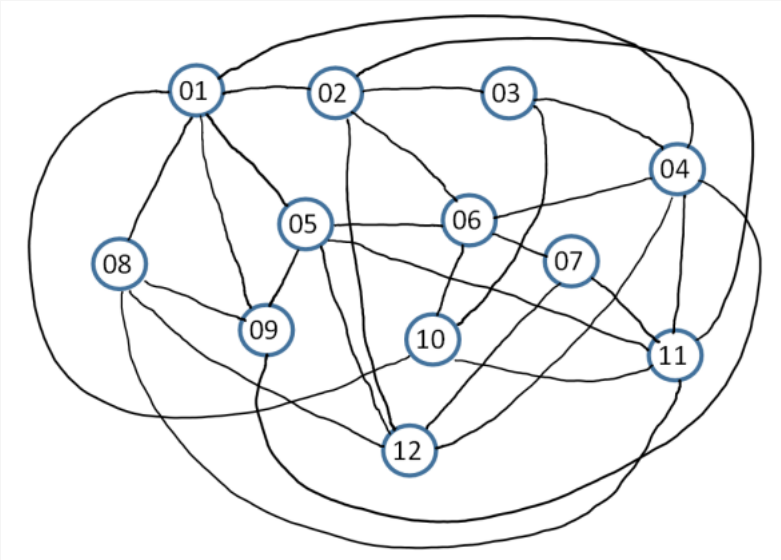
<u>capacity constraints:</u>

$$x_{SA} \le 6,\ \ x_{SB} \le 1,\ \ \dots,\ \ x_{GT} \le 12,\ \ x_{FT} \le 4.$$

<u>nonnegativity constraints:</u>

$$x_{SA},\ \ x_{SB}, \dots,\ \ x_{GT},\ \ x_{FT} \ge 0$$

Let's revisit our second example – Exam scheduling



Rules of the game:
no student can have > 1 exams on the same day

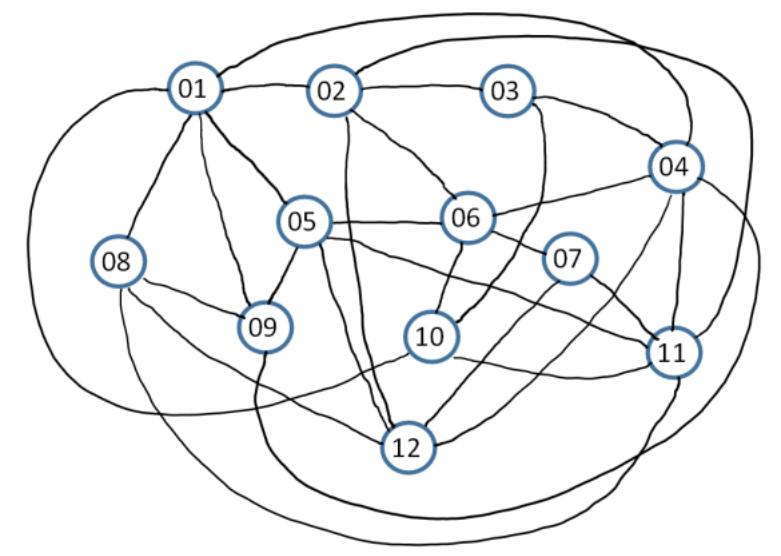
Goal: have as many exams as possible on the same day

What were your **decision variables**?

What were your **constraints**?

What was your **objective function**?

Let's revisit our second example – Exam scheduling



Decision variables:

$$x_i = \begin{cases} 1, & \text{if I chose to schedule the exam of course } i \\ 0, & \text{otherwise} \end{cases}$$

Objective: maximize the total number of scheduled exams

$$\max\ (x_1 + x_2 + \cdots + x_{12})$$

Constraints:

<u>no student should have more than 1 exam:</u>

$x_1 + x_2 \leq 1, x_1 + x_5 \leq 1,\ \ldots x_1 + x_{11} \leq 1,$
$x_2 + x_3 \leq 1, x_2 + x_6 \leq 1, \ldots, x_2 + x_{12} \leq 2,$
… (for every edge)

<u>integrality constraint:</u>

$x_i(1 - x_i) = 0,\ \ i = 1,2, \ldots, 12$

Remark: $x_i(1 - x_i) = 0$ is equivalent to $x_i \in \{0,1\}$

Why one problem is hard, and the other is easy? How can you tell?

Oil transportation problem

Decision variables:

x_{ij}: oil transported from node i to node j

$i, j \in \{S, A, B, C, D, E, F, G, T\}$

Objective: maximize the oil coming out from node S

$\max x_{SA} + x_{SB} + x_{SC}$

Constraints:

balance constraints → InFlow == OutFlow

$x_{SA} = x_{AD} + x_{AE} + x_{AB}$

$x_{AD} = x_{DG} + x_{DE}$

…

capacity constraints:

$x_{SA} \leq 6, \quad x_{AD} \leq 4, \quad x_{DG} \leq 5, \quad x_{SB} \leq 1, \quad x_{BE} \leq 20, \ldots$

nonnegativity constraints:

$x_{SA}, x_{SB}, \ldots, x_{GT}, x_{FT} \geq 0$

Exam scheduling problem

Decision variables:

$x_i = 1$, if exam of course i is held;

$x_i = 0$, otherwise

Objective: maximize the number of exams held

$\max x_1 + x_2 + \cdots + x_{12}$

Constraints:

no student should have more than 1 exam

$x_1 + x_2 \leq 1, x_1 + x_5 \leq 1, \ldots x_1 + x_{11} \leq 1,$

$x_2 + x_3 \leq 1, x_2 + x_6 \leq 1, \ldots, x_2 + x_{12} \leq 2,$

… (one constraint for every edge)

integrality constraint (ensures all $x_i \in \{0,1\}$):

$x_i(1 - x_i) = 0, \quad i = 1,2, \ldots, 12$

Fermat's last theorem

Let's see another very interesting problem:

Can you find 3 positive integers x, y, z such that

$$x^2 + y^2 = z^2 \text{ ??}$$

Fermat's last theorem

Let's see another very interesting problem:

Can you find 3 positive integers x, y, z such that

$$x^2 + y^2 = z^2 \text{ ??}$$

There are many triplets: $(3,4,5)$, $(5,12,13)$, $(8,15,17)$, $(7,24,25)$
$(20,21,29)$, $(12,35,37)$, $(9,40,41)$, $(28,45,53)$, …

There are infinitely many such triplets

Fermat's last theorem

Let's see another very interesting problem:

Can you find 3 positive integers x, y, z such that

$$x^2 + y^2 = z^2 \text{ ??}$$

There are many triplets: $(3,4,5)$, $(5,12,13)$, $(8,15,17)$, $(7,24,25)$
$(20,21,29)$, $(12,35,37)$, $(9,40,41)$, $(28,45,53)$, …

There are infinitely many such triplets

How about: $x^3 + y^3 = z^3$?

How about: $x^4 + y^4 = z^4$?

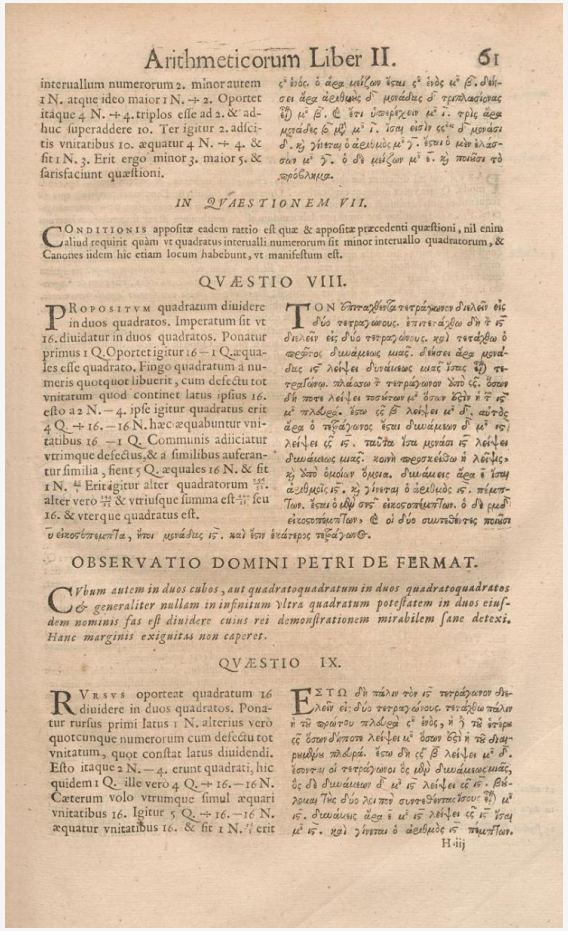
How about: $x^5 + y^5 = z^5$?

Fermat's last theorem

<u>Fermat's conjecture (1637):</u>
For $n \geq 3$, the equation $x^n + y^n = z^n$ **have no solution over the positive integers**

Pierre de Fermat claimed that: *he had a proof that was too large to fit in the margin of Diophantus' Arithmetica.*

Finally proved in 1994 (**357 years later!**) by **Andrew Wiles**.
(He was a faculty in our math department until a few years ago)



Arithmeticorum Liber II. 61

interuallum numerorum 2. minor autem 1 N. atque ideo maior 1 N. + 2. Oportet itaque 4 N. + 4. triplos esse ad 2. & adhuc superaddere 10. Ter igitur 2. adficitis vnitatibus 10. æquatur 4 N. + 4. & fit 1 N. 3. Erit ergo minor 3. maior 5. & satisfaciunt quæstioni.

IN QVAESTIONEM VII.

CONDITIONIS appositæ eadem ratio est quæ & appositæ præcedenti quæstioni, nil enim aliud requirit quàm vt quadratus interualli numerorum sit minor interuallo quadratorum, & Canones iidem hic etiam locum habebunt, vt manifestum est.

QVÆSTIO VIII.

PROPOSITVM quadratum diuidere in duos quadratos. Imperatum sit vt 16. diuidatur in duos quadratos. Ponatur primus 1 Q. Oportet igitur 16 − 1 Q. æquales esse quadrato. Fingo quadratum a numeris quotquot libuerit, cum defectu tot vnitatum quot continet latus ipsius 16. esto a 2 N. − 4. ipse igitur quadratus erit 4 Q. + 16. − 16 N. hæc æquabuntur vnitatibus 16 − 1 Q. Communis adiiciatur vtrimque defectus, & a similibus auferantur similia, fient 5 Q. æquales 16 N. & fit 1 N. $\frac{16}{5}$. Erit igitur alter quadratorum $\frac{256}{25}$. alter vero $\frac{144}{25}$ & vtriusque summa est $\frac{400}{25}$ seu 16. & vterque quadratus est.

OBSERVATIO DOMINI PETRI DE FERMAT.

CVbum autem in duos cubos, aut quadratoquadratum in duos quadratoquadratos & generaliter nullam in infinitum vltra quadratum potestatem in duos eiusdem nominis fas est diuidere cuius rei demonstrationem mirabilem sane detexi. Hanc marginis exiguitas non caperet.

QVÆSTIO IX.

RVRSVS oporteat quadratum 16 diuidere in duos quadratos. Ponatur rursus primi latus 1 N. alterius verò quotcunque numerorum cum defectu tot vnitatum, quot constat latus diuidendi. Esto itaque 2 N. − 4. erunt quadrati, hic quidem 1 Q. ille verò 4 Q. + 16. − 16 N. Cæterum volo vtrumque simul æquari vnitatibus 16. Igitur 5 Q. + 16. − 16 N. æquatur vnitatibus 16. & fit 1 N. $\frac{16}{5}$ erit

H iij

Diophantus' *Arithmetica*
(3rd century AD)

Fermat's last theorem

Fermat's conjecture (1637):
For $n \geq 3$, the equation $x^n + y^n = z^n$ **have no solution over the positive integers**

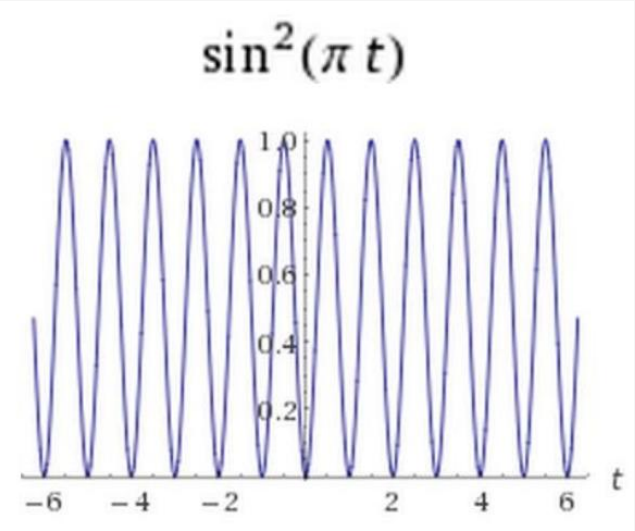
It can be formulated as an optimization problem!

What are decision variables, objective and constraints???

Decision variables: x, y, z, n

Objective: $\boldsymbol{min}(x^n + y^n - z^n)^2$

Constraints: $\sin^2(\pi x) + \sin^2(\pi y) + \sin^2(\pi z) = 0$
$x \geq 1, \quad y \geq 1, \quad z \geq 1$ and $n \geq 3$



Innocent looking optimization problem: **only 4 variables and 5 constraints.**

If you could prove that the optimal value is <u>always non-zero</u>, you would prove Fermat's conjecture!

Course objectives

The skills you will acquire:

- Ability to **formulate and solve** problems in your field of study as Optimization problems.
- Focus on problems arising in: **Operations Research, Statistics, Finance, Machine Learning, etc**.
- Learn about several topics in **scientific computing.**
- Gain more **mathematical maturity** and ability for rigorous reasoning.
- **Enhance your coding abilities** (nothing too fancy, simple Matlab or Python)
  >> there will be computational problems in every **homework** and on **take-home final exam.**
- Ability to recognize **hard and easy optimization problems.**
- Ability to use **optimization software** (CVX solver) and understand the algorithms they use.

Software you need to download and know

- If you choose to use **Matlab**, you can download the code from the OIT website
  - https://kb.princeton.edu/KB0011341

- If you choose to use **Python**, you can download it for free from various places (e.g., Anaconda)

- The solutions will be provided in both **Matlab** and **Python**

- You also have to be familiar with **CVX solver**, the optimization solver we will be using in the course
  - you can download it for free
    - for **Matlab** check: http://cvxr.com/cvx
    - for **Python** check: http://www.cvxpy.org

- Computational problems will appear after HW2 or HW3 (in the next couple of weeks).

- Some useful links:
  - for **Matlab**: https://www.mathworks.com/help/matlab/getting-started-with-matlab.html
  - for **Python**: https://pythonnumericalmethods.berkeley.edu/notebooks/Index.html (***)

Course grades

- **50% homework assignments (collaboration is encouraged!)**
  - around 8 problem sets; will drop the lowest score;
  - almost all problem sets will involve a computational component.
- **20% midterm exam**
  - in class, during regular lecture time, closed notes (except for one page of notes)
- **30% final exam**
  - 48-hour take-home, open notes, involves a computational component

Course Syllabus

Always check the course **Syllabus** for all requirements and rules!

See: `Canvas/Files/Syllabus`

Some useful textbooks (for reference)

- Textbooks Recommended as reference (instructor lecture notes are the only required reference):
  - **An Introduction to Optimization**, K. P. Chong & S. H. Zak, 4th ed.
  - **Algorithms**, S. Dasgupta, C. Papadimitriou, and U. Vazirani.
  - **Introduction to Applied Linear Algebra – Vectors, Matrices, and Least Squares** https://web.stanford.edu/~boyd/vmls/
  - **Linear Programming: Foundations and Extensions**, R.J. Vanderbei. Available for free download to Princeton students: http://link.springer.com/book/10.1007/978-1-4614-7630-6
  - **Convex Optimization**, S.Boyd, and L. Vandenberghe. Available for free download: http://web.stanford.edu/~boyd/cvxbook/

References used in this lecture

- [DPC08] S. Dasgupta, C. Papadimitriou, U. Vazirani, “Algorithms”, McGraw Hill, 2008
- [Sch05] A. Schrijver, “On the history of combinatorial optimization (till 1960)”, in “Handbook of discrete optimization”, Elsevier, 2005.
  - Available online: http://homepages.cwi.nl/~lex/files/histco.pdf